

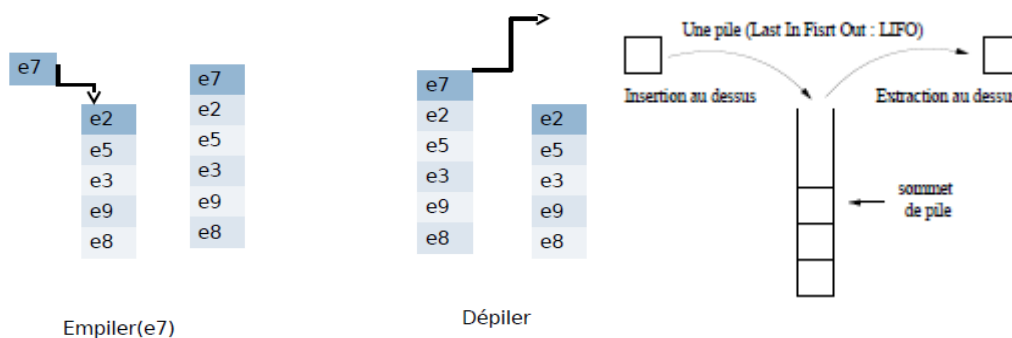
CLASSES PREPARATOIRES AUX GRANDES ECOLES : CPGE Option : Physique Chimie et Sciences de l'Ingénieur (PCSI sup)		
Etablissement : CPGE ISMAILIA	Pile et file SERIE N°10	Enseignante : L.OUSTOUH

Cours : PILE

Une pile (en anglais *stack*) est une structure de données fondée sur le principe « dernier arrivé, premier sorti » (ou LIFO pour Last In, First Out)

– Les derniers éléments ajoutés à la pile seront les premiers à être récupérés. Exemple : Pile d'assiettes : on ajoute des assiettes sur la pile, et on les récupère dans l'ordre inverse, en commençant par la dernière ajoutée.

Représentation :



TD : PILE

Exercice 1 :

On implémentera une pile en envisageant un stockage en tableau global. Les paramètres comme la taille maximum de la pile sera déclarée constante. Enfin le sommet de la pile sera également déclaré en variable globale, il pointerait sur le dernier élément empilé.

Ecrire les fonctions/procédures suivantes :

1. `creerPile()`
2. `afficherPile()` ;
3. `estVidePile()` ;
4. `estPleinePile()` ;
5. `sommetPile()` ;
6. `empilerPile( e :entier)` ;
7. `depilerPile()` ;

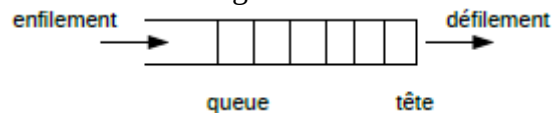
Indication :

- `creerPile` attribut au sommet de la pile la valeur -1
- `afficherPile` ne retourne rien ;
- `estVidePile` retourne un entier (1 : vrai si la pile est vide, 0 : faux sinon) ;
- `estPleinePile` retourne un entier (1 : vrai si la pile est pleine, 0 : faux sinon) ;
- `sommetPile` retourne un entier (le sommet de la pile) ;
- `empilerPile` prend un entier à empiler et ne retourne rien ;
- `depilerPile` retourne l'élément dépilé (le sommet).

Cours : LES FILES (FIFO)

une **file** (en anglais *queue*) est une structure de données basée sur le principe « premier arrive, premier sorti », en anglais FIFO (First In, First Out) : Les premiers éléments ajoutés à la file

seront les premiers à être récupérés. Dans ce type de liste, on effectue l'insertion à la queue de la liste et la suppression à la tête. Exemple : file devant un guichet.



La représentation mémoire adoptée est :

Plusieurs approches existent pour implémenter une file :

Dans toutes les approches le nombre maximum N d'éléments est fixé.

1. Première approche : par décalage

tête = l'indice 0 (fixe)

queue = l'indice du dernier elts

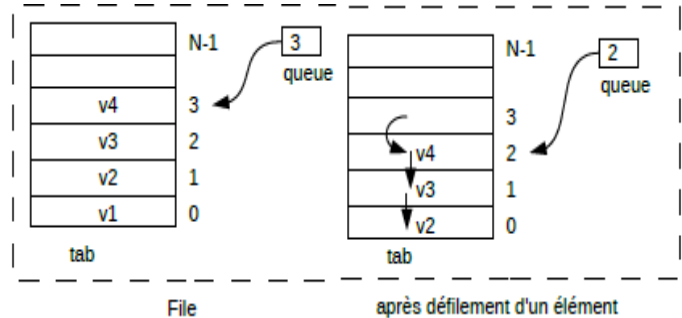
défiler = décalages vers le bas

(queue ← queue-1)

enfiler : (queue ← queue+1)

FileVide : (queue == -1)

FilePleine : (queue == N-1)



2. Deuxième approche : par tableaux circulaires

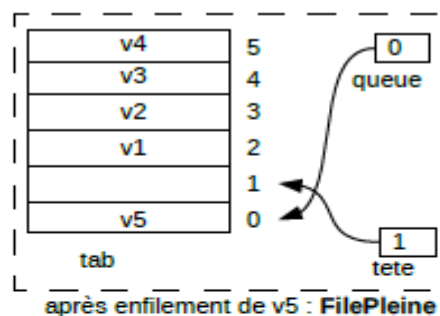
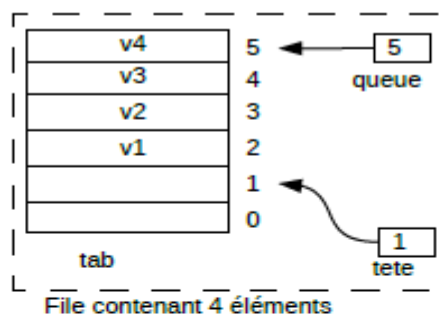
tête = s'incrémente modulo N à chaque défilement

queue = s'incrémente modulo N à chaque enfilement.

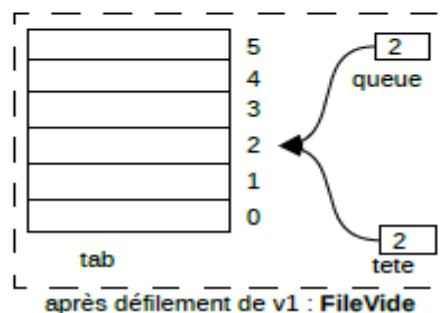
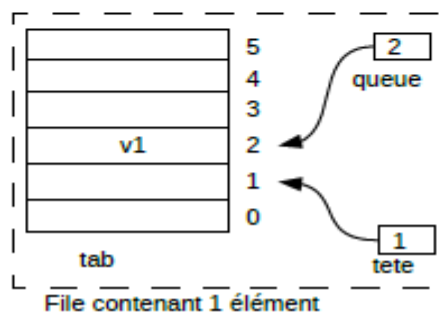
Filevide = tete==queue

Filepleinr=tete==(queue+1) modulo N

Explication :



FilePleine ssi :
 $tete = (queue + 1 \bmod N)$



FileVide ssi :
 $tete = queue$

Exercice :

Implémenter les fonctions/procédure de gestion de file suivante :

Procédure creerFile()

Fonction fileVide() :entier

Fonction filePleine() :entier

Procédure enfiler(x :entier)

Fonction defiler() :entier

Procédure afficherFile()
Fonction premierFile() :entier

Problème :

On s'intéresse au traitement des expressions arithmétiques ou logiques utilisées dans les programmes écrits dans un langage de programmation classique : les algorithmes d'évaluation et de transformation d'écriture d'expressions nécessitent l'utilisation d'une pile. L'expression peut être écrite :

- Sous forme complètement parenthésée (ECP) $((A+B) * C)$
- Sous forme préfixée (EPR) $* + A B C$
- Sous forme postfixée (EPO) ou (notation polonaise inversée) $A B + C *$
- Sous forme infixée (EIN) $(A + B) * C$

Evaluer une expression consiste à calculer la valeur de l'expression pour les valeurs associées à chaque variable.

On supposera que l'expression est terminée par # et de taille $\leq \text{MAX}$. On se limitera aux variables réelles identifiées par un seul caractère et aux 4 opérateurs '+', '-', '*', et '/'.

1. Écrire une fonction qui permet d'inverser une pile, on supposera que la pile et le sommet sont des variables globales.
2. Écrire une fonction qui prend en entrée une expression mathématique simple stockée dans un tableau de caractères et composée de différentes parenthèses, crochets, ouvrants ou fermants, et qui vérifie que l'expression est bien parenthésée.

Ainsi, cette fonction doit :

- accepter les expressions contenant (), ()() ou ((())())
- rejeter les expressions contenant) (, ()(ou (((()))

La fonction doit analyser, un par un, les caractères du tableau.

Rappels :

- on peut accéder au i ème caractère d'un tableau str de la façon suivante :
str[i].

L'analyse se fait comme décrit ci-après.

Pour chacun des caractères :

- si c'est une parenthèse ouvrante '(', une fermante est empilée ;
- si c'est une parenthèse ')' fermante, sa correspondante dans la pile est dépilée.

L'expression est acceptée si :

- la pile n'est jamais vide à la lecture d'une fermante ; et si
- la pile est vide lorsque toute l'expression a été analysée.

On pourra par exemple suivre les étapes suivantes :

- une fonction qui prend en entrée un caractère et renvoie vrai s'il s'agit d'un symbole ouvrant et faux sinon,
- une fonction qui prend un symbole ouvrant et fournit en retour le caractère fermant correspondant,
- enfin, la procédure de vérification, on supposera que les fonctions et procédures de gestion de pile sont implémentés.

3. La notation polonaise inversée est une façon d'écrire les expressions arithmétiques sans indiquer de multiples parenthèses. Une expression arithmétique écrite en notation polonaise inversée est une suite de nombres et d'opérateurs (+, -, / ou *) et se lit (et s'évalue) de gauche à droite de la façon suivante :

– lorsqu'un nombre est lu, il est empilé sur une pile (initialement vide) ;

– lorsqu'un opérateur est lu, les deux premiers éléments de la pile sont récupérés et dépilés, l'opération est effectuée sur les deux valeurs obtenues, et le résultat est empilé ;
– le résultat de l'expression est le premier (et normalement le seul) élément de la pile à la fin de la lecture.

1. Appliquer la méthode exposée pour calculer : $17\ 10\ -\ ,\ 3\ 28\ 7\ /\ +$ et $3\ 28\ +\ 7\ /\$
Comment s'écrivent ces expressions en notation classique ?

2. Traduire en notation polonaise inversée les expressions suivantes : $(19 * 6) - 7$
et $(4 - 2 * (7 + 6)) + 3 * 5$

3. Écrire une fonction qui évalue une expression polonaise inversée, composée d'entiers entre 0 et 9 et des quatre opérations élémentaires.

On pourra par exemple suivre les étapes suivantes :

- une fonction qui prend en entrée un caractère et renvoie vrai s'il s'agit d'une opération élémentaire et faux sinon,
- une fonction qui prend un symbole-opération et deux entiers et qui renvoie le résultat de l'opération,
- enfin, la fonction d'évaluation.

4. testez la fonction pour calculer : $17\ 10\ -\ ,\ 3\ 28\ 7\ /\ +$ et $3\ 28\ +\ 7\ /\$